Лабораторная работа № 4. Параллельное умножение матриц с использованием OpenMP

Шамсутдинов Ренат, группа 11-401

1. Реализация и условия измерений

Написана программа на C, в которой реализованы все пять вариантов: **ijk_seq, ijk_omp, ikj_seq, ikj_omp, blocked_omp**. В ijk_omp и ikj_omp директивой parallel for распараллелен внешний цикл по строкам i; в blocked_omp (parallel for collapse(2)) потоки получают независимые блоки матрицы C, поэтому два потока не пишут в один элемент. Число потоков, размер матрицы и размер блока вводятся с клавиатуры. Матрицы double выделяются динамически, A и B заполняются по формуле от индексов; перед каждым запуском C обнуляется; измеряется только вычисление (omp_get_wtime). Результаты всех вариантов совпали с ijk_seq (максимальная разница 0, проверка OK).

Компьютер: ноутбук, AMD Ryzen 7 5700U (8 ядер, 16 логических процессоров, база 1,8 ГГц; L1 512 КБ, L2 4 МБ, L3 8 МБ), ОЗУ 15,3 ГБ, Windows 11, GCC 16.1.0 (MSYS2), gcc -O3 -fopenmp. В ходе измерений ноутбук в какой-то момент был отключён от зарядки, поэтому время последовательных версий в разных запусках различается (ijk_seq при N = 1024: 13,7–19,1 с). Ускорение считалось относительно последовательной версии из того же запуска.

Таблица 1. Время, с, при разных N (4 потока, блок 32)

N	ijk_seq	ijk_omp	ikj_seq	ikj_omp	blocked_omp
128	0,0044	0,0018	0,0012	0,0004	0,0010
256	0,0648	0,0170	0,0074	0,0022	0,0036
512	1,6264	0,4020	0,0594	0,0164	0,0290
1024	13,9527	4,0610	0,5983	0,1970	0,2543

Время самого умножения, среднее по 5 повторам (N = 1024: по 3).

Таблица 3. Блок (N = 1024, 4 потока)

Блок	T blocked, с	T ikj_seq / T blocked	T ikj_omp, с
16	0,5710	1,51	0,2690
32	0,4447	1,98	0,2817
64	0,5303	1,47	0,2760

Отношение включает и потоки, и блочность, это не чистое ускорение.

Таблица 2. Число потоков (N = 1024, блок 32). $S = T_{seq}/T_{omp}(p)$, $E = S/p$; для blocked_omp в последнем столбце сравнение с ikj_seq

Потоки	T ijk_omp, с	S ijk	E ijk	T ikj_omp, с	S ikj	E ikj	T blocked_omp, с	T ikj_seq / T blocked
1	14,2227	0,96	0,96	0,8150	0,98	0,98	1,4243	0,56
2	9,4890	1,99	0,99	0,4370	1,79	0,90	0,7797	1,00
4	5,3743	3,52	0,88	0,2677	2,86	0,72	0,3847	1,99
8	3,0387	6,24	0,78	0,1633	4,68	0,59	0,2690	2,84
16	3,0060	6,31	0,39	0,1573	5,03	0,31	0,1700	4,65

Последовательные версии в тех же запусках (для 1, 2, 4, 8, 16 потоков): ijk_seq 13,72 / 18,85 / 18,91 / 18,95 / 18,98 с; ikj_seq 0,799 / 0,783 / 0,766 / 0,765 / 0,791 с.

2. Выводы

- **ikj намного быстрее ijk даже на одном потоке.** По таблице 1 ikj_seq быстрее ijk_seq в 3,7 раза при N = 128, в 8,8 при 256, в 27 при 512 и в 23 при 1024. Разрыв растёт с размером матрицы: в ijk внутренний цикл читает B по столбцам с шагом N, и когда B перестаёт помещаться в кэш, почти каждое обращение уходит в память. В ikj и B, и C читаются подряд. При 16 потоках ikj_omp быстрее ijk_omp примерно в 19 раз.
- **Многопоточность.** До 4 потоков ускорение растёт почти линейно у ijk_omp (S = 3,52, E = 0,88) и заметно слабее у ikj_omp (S = 2,86, E = 0,72). Максимум S = 6,3 у ijk_omp и 5,0 у ikj_omp. ijk масштабируется лучше, вероятно, потому что медленные обращения к памяти у разных потоков перекрываются; ikj уже быстро использует кэш и раньше упирается в общую пропускную способность L3 и памяти.
- **Где ускорение перестало расти:** после 8 потоков. Переход с 8 на 16 почти не меняет время (ijk_omp 3,04 → 3,01 с, ikj_omp 0,163 → 0,157 с), эффективность падает до 0,39 и 0,31. В процессоре 8 физических ядер, а 16 логических потоков (SMT) делят исполнительные устройства и кэши ядра.
- **Блочное умножение.** Лучший размер блока 32 (0,445 с против 0,571 при 16 и 0,530 при 64). Три блока 32×32 типа double занимают 24 КБ и помещаются в L1-кэш данных (32 КБ на ядро), три блока 64×64 (96 КБ) уже нет; при блоке 16 больше накладных расходов на циклы. Однако blocked_omp не обогнал ikj_omp (при 4 потоках 0,385 с против 0,268 с, при 16 потоках 0,170 с против 0,157 с): у этого процессора большие L2 и L3, ikj уже хорошо использует кэш, а короткий внутренний цикл блока хуже векторизуется (вывод предположительный).
- **Ограничения.** 8 физических ядер и SMT; общая память и кэш L3; ноутбучный процессор с ограничением мощности и частоты; отключение от зарядки во время серии замеров (разброс времени ijk_seq); при N = 128 время 0,4–4 мс сопоставимо с накладными расходами на создание потоков, а число повторов невелико, поэтому небольшие различия в таблицах могут быть шумом.